

Software Requirements Specification

Digital Visitor Feedback and Experience Management Portal

Document Type:	Software Requirements Specification (SRS)
Category:	Vibe Coding & Full Stack Development
Difficulty Level:	Medium
Version:	1.0
Prepared For:	24-Hour Hackathon Submission

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Digital Visitor Feedback and Experience Management Portal**. It describes the purpose, scope, functional and non-functional requirements, system modules, and constraints for the system to be developed during the 24-hour hackathon. This SRS serves as a reference for the development team, mentors, and evaluators to understand what the system will and will not do.

1.2 Problem Statement

Design and develop a comprehensive portal for digital visitor feedback and experience management that digitizes manual processes, provides role-based access for administrators, staff, and users, automates workflows, generates analytical reports, sends real-time notifications, and integrates securely with institutional systems to improve operational efficiency, transparency, collaboration, and user experience.

1.3 Project Overview (In Simple Terms)

Many institutions (hospitals, universities, government offices, corporate campuses, museums, etc.) still collect visitor feedback through paper forms or physical suggestion boxes, which are slow to process, difficult to track, and easy to ignore. This project replaces that manual process with a web-based portal where visitors can submit feedback digitally, staff can manage and resolve issues through a structured workflow, and administrators can monitor overall performance through analytics dashboards. The system is entirely rule-based and deterministic; it does not use any Artificial Intelligence or Machine Learning components.

1.4 Scope

- Multi-role web application supporting Visitor, Staff, and Admin roles.
- Digital feedback submission and status tracking system.
- Rule-based workflow automation for feedback routing and escalation.
- Real-time notification system for status updates.
- Analytics and reporting dashboard using data aggregation (no AI/ML).
- Secure integration layer with external/institutional systems (mocked for hackathon scope).

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
JWT	JSON Web Token, used for secure authentication
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
UI/UX	User Interface / User Experience

2. Overall Description

2.1 Product Perspective

The system is a standalone, self-contained web application composed of a frontend client, a backend API server, a database, and supporting services for notifications and analytics. It is designed to be deployed independently and to optionally integrate with an institution's existing systems through a secure API/webhook layer.

2.2 User Classes and Characteristics

Role	Description	Key Permissions
Visitor / User	General public or institutional visitors who interact with the institution.	Register/login, submit feedback, upload attachments, track status, receive notifications.
Staff	Employees responsible for reviewing and resolving feedback.	View assigned feedback, update status, respond to visitors, view team-level analytics.
Admin	System administrators overseeing the entire portal.	Manage users and roles, configure workflow rules, view organization-wide analytics, manage integrations.

2.3 Operating Environment

- Client Side: Any modern web browser (Chrome, Firefox, Edge, Safari), responsive for desktop and mobile.
- Server Side: Node.js runtime environment hosted on a cloud platform (e.g., Render, Vercel).
- Database: Cloud-hosted relational or NoSQL database (e.g., PostgreSQL/MongoDB via Supabase).

2.4 Design and Implementation Constraints

- The system must be built and deployed within a 24-hour hackathon timeframe.
- No Artificial Intelligence or Machine Learning components are permitted in any module; all automation and analytics must be rule-based and deterministic.
- Institutional system integration will be simulated (mock API/webhook or CSV import) due to lack of access to real institutional systems during the hackathon.
- Must use free-tier or open-source tools/services suitable for rapid development.

2.5 Assumptions and Dependencies

- Users have access to a stable internet connection and a modern web browser.
- The institution's real backend systems are not accessible; integration is demonstrated via a mock endpoint.
- Email/notification services used (e.g., Resend, SendGrid, Supabase Realtime) provide free-tier access sufficient for demo purposes.

3. System Modules

The system is divided into ten core modules. All modules are rule-based and deterministic — no AI/ML components are used anywhere in the system.

#	Module	Type	Description
1	User Management	CRUD	Handles registration, login, logout, and profile management for all user roles.
2	Authentication & Authorization	Security	JWT-based session handling and Role-Based Access Control (RBAC) restricting pages/routes per role.
3	Feedback Submission	CRUD + Forms	Feedback form with category selection, rating, description, and file attachment; generates unique feedback ID and timestamp.
4	Workflow Automation	Rule-based logic	Auto-assigns feedback to relevant staff/department using if-else/lookup rules; escalates unresolved items after a defined time threshold.
5	Notification System	Event-triggered	Sends real-time in-app and/or email notifications on new feedback, status changes, and escalations.
6	Analytics & Reporting	Data aggregation	Dashboard with charts built from SQL aggregation (AVG, COUNT, GROUP BY) — category breakdown, satisfaction trend, resolution time.
7	Admin Panel	CRUD + Config	Enables admins to manage users/roles, configure workflow rules, and view organization-wide analytics.
8	Integration Layer	API/Webhook	Secure API endpoint or webhook for connecting with institutional systems (mocked for hackathon scope).
9	Database	Schema/Storage	Stores Users, Feedback, Roles, Notifications, and Logs in a structured schema.
10	Security	Validation/Encryption	Input validation, password hashing, HTTPS enforcement, and secure session handling across the system.

Note: No AI/ML module is included. Feedback categorization is performed via manual dropdown selection by the visitor rather than automatic text classification, and all analytics are based on statistical aggregation, not predictive modeling.

4. Functional Requirements

ID	Requirement	Related Module
FR1	The system shall allow users to register and log in with role-based credentials.	User Management / Auth
FR2	The system shall allow visitors to submit feedback with category, rating, description, and optional attachment.	Feedback Submission
FR3	The system shall track feedback status through defined states: Open, In Progress, Resolved, Closed.	Feedback Submission
FR4	The system shall automatically assign incoming feedback to the relevant staff/department based on category rules.	Workflow Automation
FR5	The system shall automatically escalate feedback that remains unresolved beyond a configurable time threshold.	Workflow Automation
FR6	The system shall send real-time notifications to staff on new feedback and to visitors on status updates.	Notification System
FR7	The system shall display an analytics dashboard showing category-wise trends, satisfaction scores, and resolution times.	Analytics & Reporting
FR8	The system shall provide an admin panel for managing users, roles, and workflow configuration.	Admin Panel
FR9	The system shall expose a secure API/webhook endpoint to simulate integration with institutional systems.	Integration Layer
FR10	The system shall allow staff and admins to search and filter feedback records by status, category, and date.	Feedback Submission

5. Non-Functional Requirements

Category	Requirement
Security	All passwords shall be hashed; role-based access control shall be enforced on every API endpoint; sessions secured via JWT.
Usability	The interface shall be responsive and usable on both desktop and mobile browsers.
Performance	The system shall handle concurrent feedback submissions without data loss and load core pages within 2-3 seconds under normal conditions.
Reliability	Feedback data shall be persisted reliably with no loss on submission, even under network interruptions (client-side retry).
Scalability	The backend shall follow a modular structure allowing new modules or institutional integrations to be added easily.
Maintainability	Code shall be organized by module with clear separation between frontend, backend, and database layers.

6. Proposed System Architecture

The system follows a standard three-tier web architecture consisting of a presentation layer (frontend), an application layer (backend API with business/workflow logic), and a data layer (database). Notifications and the integration layer operate as supporting services connected to the application layer.

- Frontend (React + Tailwind CSS) — visitor, staff, and admin interfaces
- Backend API (Node.js / Express) — authentication, business logic, workflow automation
- Database (PostgreSQL / MongoDB) — Users, Feedback, Roles, Notifications, Logs
- Notification Service — real-time / email alerts triggered by backend events
- Integration Layer — secure API/webhook for institutional system connectivity (mocked)

6.1 Proposed Technology Stack

Layer	Technology
Frontend	React (Vite) + Tailwind CSS
Backend	Node.js + Express (or Next.js API routes)
Database	PostgreSQL / MongoDB (Supabase recommended)
Authentication	JWT / Supabase Auth
Real-time Notifications	Supabase Realtime / Socket.io
Deployment	Vercel (frontend) + Supabase / Render (backend & DB)

6.2 Feedback Status Workflow

Open → In Progress → Resolved → Closed, with an automatic Escalated state triggered if an item remains Open or In Progress beyond a configurable time threshold (e.g., 48 hours).

7. Out of Scope / Future Enhancements

- Real integration with live institutional systems (beyond the hackathon's mock endpoint).
- Multi-language support for feedback forms.
- SMS-based notifications in addition to in-app/email.
- Advanced analytics such as sentiment analysis (explicitly excluded per mentor guidance — no AI/ML).

End of Document — Software Requirements Specification v1.0